11.1 Binary Search Trees

In Chapter 8 we introduced the tree data structure and demonstrated a variety of applications. One important use is as a ***search tree*** (as described on page 332). In this chapter, we use a search tree structure to efficiently implement a ***sorted map***. The three most fundamental methods of a map M (see Section 10.1.1) are:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError; implemented with __getitem__ method.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k; implemented with __setitem__ method.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError; implemented with __delitem__ method.

The sorted map ADT includes additional functionality (see Section 10.3), guaranteeing that an iteration reports keys in sorted order, and supporting additional searches such as find_gt(k) and find_range(start, stop).

Binary trees are an excellent data structure for storing items of a map, assuming we have an order relation defined on the keys. In this context, a ***binary search tree*** is a binary tree T with each position p storing a key-value pair (k, v) such that:

- Keys stored in the left subtree of p are less than k.
- Keys stored in the right subtree of p are greater than k.

An example of such a binary search tree is given in Figure 11.1. As a matter of convenience, we will not diagram the values associated with keys in this chapter, since those values do not affect the placement of items within a search tree.

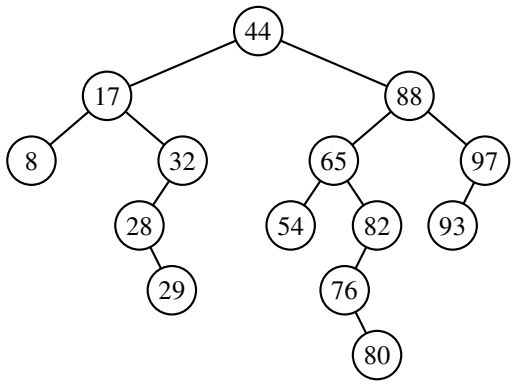


Figure 11.1: A binary search tree with integer keys. We omit the display of associated values in this chapter, since they are not relevant to the order of items within a search tree.

11.1.1 Navigating a Binary Search Tree

We begin by demonstrating that a binary search tree hierarchically represents the sorted order of its keys. In particular, the structural property regarding the placement of keys within a binary search tree assures the following important consequence regarding an ***inorder traversal*** (Section 8.4.3) of the tree.

Proposition 11.1: *An inorder traversal of a binary search tree visits positions in increasing order of their keys.*

Justification: We prove this by induction on the size of a subtree. If a subtree has at most one item, its keys are trivially visited in order. More generally, an inorder traversal of a (sub)tree consists of a recursive traversal of the (possibly empty) left subtree, followed by a visit of the root, and then a recursive traversal of the (possibly empty) right subtree. By induction, a recursive inorder traversal of the left subtree will produce an iteration of the keys in that subtree in increasing order. Furthermore, by the binary search tree property, all keys in the left subtree have keys strictly smaller than that of the root. Therefore, visiting the root just after that subtree extends the increasing order of keys. Finally, by the search tree property, all keys in the right subtree are strictly greater than the root, and by induction, an inorder traversal of that subtree will visit those keys in increasing order. ■

Since an inorder traversal can be executed in linear time, a consequence of this proposition is that we can produce a sorted iteration of the keys of a map in linear time, when represented as a binary search tree.

Although an inorder traversal is typically expressed using a top-down recursion, we can provide nonrecursive descriptions of operations that allow more fine-grained navigation among the positions of a binary search relative to the order of their keys. Our generic binary tree ADT from Chapter 8 is defined as a positional structure, allowing direct navigation using methods such as `parent(p)`, `left(p)`, and `right(p)`. With a binary *search* tree, we can provide additional navigation based on the natural order of the keys stored in the tree. In particular, we can support the following methods, akin to those provided by a `PositionalList` (Section 7.4.1).

first( ): Return the position containing the least key, or `None` if the tree is empty.

last( ): Return the position containing the greatest key, or `None` if empty tree.

before(p): Return the position containing the greatest key that is less than that of position `p` (i.e., the position that would be visited immediately before `p` in an inorder traversal), or `None` if `p` is the first position.

after(p): Return the position containing the least key that is greater than that of position `p` (i.e., the position that would be visited immediately after `p` in an inorder traversal), or `None` if `p` is the last position.

The "first" position of a binary search tree can be located by starting a walk at the root and continuing to the left child, as long as a left child exists. By symmetry, the last position is reached by repeated steps rightward starting at the root.

The successor of a position, after(p), is determined by the following algorithm.

```
Algorithm after(p):
    if right(p) is not None then {successor is leftmost position in p's right subtree}
      walk = right(p)
      while left(walk) is not None do
        walk = left(walk)
      return walk
    else {successor is nearest ancestor having p in its left subtree}
      walk = p
      ancestor = parent(walk)
      while ancestor is not None and walk == right(ancestor) do
        walk = ancestor
        ancestor = parent(walk)
      return ancestor
```

Code Fragment 11.1: Computing the successor of a position in a binary search tree.

The rationale for this process is based purely on the workings of an inorder traversal, given the correspondence of Proposition 11.1. If p has a right subtree, that right subtree is recursively traversed immediately after p is visited, and so the first position to be visited after p is the *leftmost* position within the right subtree. If p does not have a right subtree, then the flow of control of an inorder traversal returns to p's parent. If p were in the *right* subtree of that parent, then the parent's subtree traversal is complete and the flow of control progresses to its parent and so on. Once an ancestor is reached in which the recursion is returning from its *left* subtree, then that ancestor becomes the next position visited by the inorder traversal, and thus is the successor of p. Notice that the only case in which no such ancestor is found is when p was the rightmost (last) position of the full tree, in which case there is no successor.

A symmetric algorithm can be defined to determine the predecessor of a position, before(p). At this point, we note that the running time of single call to after(p) or before(p) is bounded by the height h of the full tree, because it is found after either a single downward walk or a single upward walk. While the worst-case running time is $O(h)$, we note that either of these methods run in $O(1)$ ***amortized*** time, in that series of n calls to after(p) starting at the first position will execute in a total of $O(n)$ time. We leave a formal justification of this fact to Exercise C-11.34, but intuitively the upward and downward paths mimic steps of the inorder traversal (a related argument was made in the justification of Proposition 9.3).

11.1.2 Searches

The most important consequence of the structural property of a binary search tree is its namesake search algorithm. We can attempt to locate a particular key in a binary search tree by viewing it as a decision tree (recall Figure 8.7). In this case, the question asked at each position p is whether the desired key k is less than, equal to, or greater than the key stored at position p, which we denote as p.key( ). If the answer is "less than," then the search continues in the left subtree. If the answer is "equal," then the search terminates successfully. If the answer is "greater than," then the search continues in the right subtree. Finally, if we reach an empty subtree, then the search terminates unsuccessfully. (See Figure 11.2.)

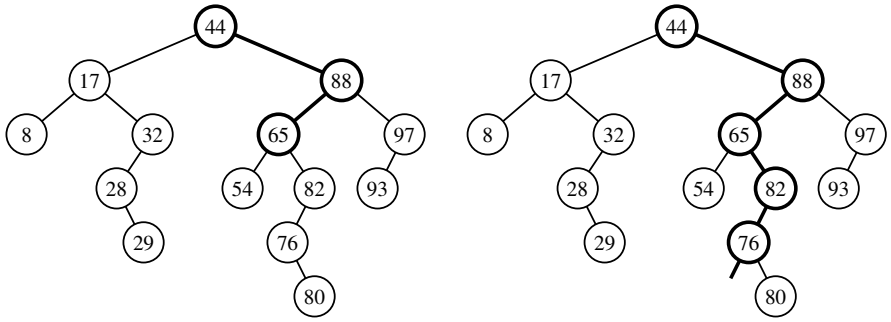


Figure 11.2: (a) A successful search for key 65 in a binary search tree; (b) an unsuccessful search for key 68 that terminates because there is no subtree to the left of the key 76.

We describe this approach in Code Fragment 11.2. If key k occurs in a subtree rooted at p, a call to TreeSearch(T, p, k) results in the position at which the key is found; in this case, the __getitem__ map operation would return the associated value at that position. In the event of an unsuccessful search, the TreeSearch algorithm returns the final position explored on the search path (which we will later make use of when determining where to insert a new item in a search tree).

```
Algorithm TreeSearch(T, p, k):
    if k == p.key() then
        return p                                        {successful search}
    else if k < p.key() and T.left(p) is not None then
        return TreeSearch(T, T.left(p), k)              {recur on left subtree}
    else if k > p.key() and T.right(p) is not None then
        return TreeSearch(T, T.right(p), k)             {recur on right subtree}
    return p                                            {unsuccessful search}
```

Code Fragment 11.2: Recursive search in a binary search tree.

Analysis of Binary Tree Searching

The analysis of the worst-case running time of searching in a binary search tree T is simple. Algorithm TreeSearch is recursive and executes a constant number of primitive operations for each recursive call. Each recursive call of TreeSearch is made on a child of the previous position. That is, TreeSearch is called on the positions of a path of T that starts at the root and goes down one level at a time. Thus, the number of such positions is bounded by $h+1$, where h is the height of T. In other words, since we spend $O(1)$ time per position encountered in the search, the overall search runs in $O(h)$ time, where h is the height of the binary search tree T. (See Figure 11.3.)

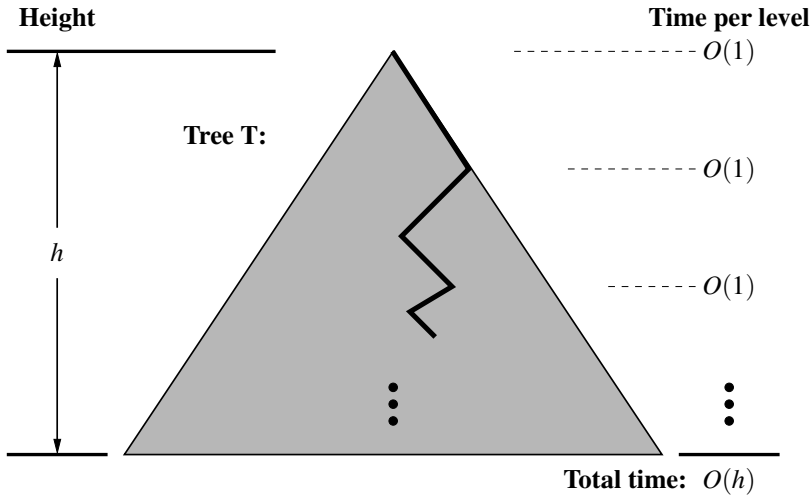


Figure 11.3: Illustrating the running time of searching in a binary search tree. The figure uses standard caricature of a binary search tree as a big triangle and a path from the root as a zig-zag line.

In the context of the sorted map ADT, the search will be used as a subroutine for implementing the __getitem__ method, as well as for the __setitem__ and __delitem__ methods, since each of these begins by trying to locate an existing item with a given key. To implement sorted map operations such as find_lt and find_gt, we will combine this search with traversal methods before and after. All of these operations will run in worst-case $O(h)$ time for a tree with height h. We can use a variation of this technique to implement the find_range method in time $O(s+h)$, where s is the number of items reported (see Exercise C-11.34).

Admittedly, the height h of T can be as large as the number of entries, n, but we expect that it is usually much smaller. Indeed, later in this chapter we show various strategies to maintain an upper bound of $O(\log n)$ on the height of a search tree T.

11.1.3 Insertions and Deletions

Algorithms for inserting or deleting entries of a binary search tree are fairly straightforward, although not trivial.

Insertion

The map command M[k] = v, as supported by the __setitem__ method, begins with a search for key k (assuming the map is nonempty). If found, that item's existing value is reassigned. Otherwise, a node for the new item can be inserted into the underlying tree T in place of the empty subtree that was reached at the end of the failed search. The binary search tree property is sustained by that placement (note that it is placed exactly where a search would expect it). Pseudo-code for such a TreeInsert algorithm is given in in Code Fragment 11.3.

Algorithm TreeInsert(T, k, v):
Input: A search key k to be associated with value v
p = TreeSearch(T, T.root(), k)
if k == p.key() **then**
  Set p's value to v
else if k < p.key() **then**
  add node with item (k,v) as left child of p
else
  add node with item (k,v) as right child of p

Code Fragment 11.3: Algorithm for inserting a key-value pair into a map that is represented as a binary search tree.

An example of insertion into a binary search tree is shown in Figure 11.4.

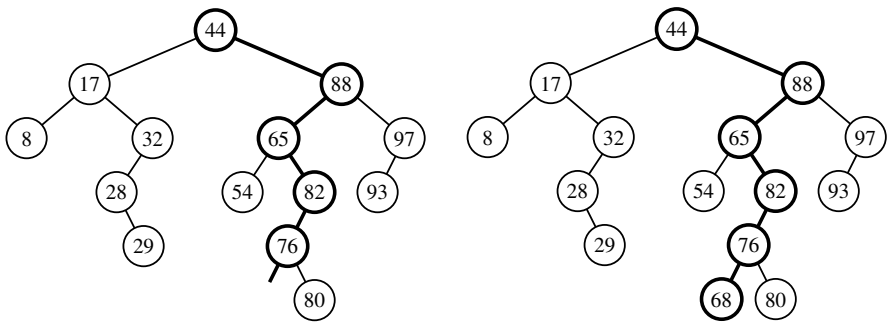


Figure 11.4: Insertion of an item with key 68 into the search tree of Figure 11.2. Finding the position to insert is shown in (a), and the resulting tree is shown in (b).

Deletion

Deleting an item from a binary search tree T is a bit more complex than inserting a new item because the location of the deletion might be anywhere in the tree. (In contrast, insertions are always enacted at the bottom of a path.) To delete an item with key k, we begin by calling TreeSearch(T, T.root( ), k) to find the position p of T storing an item with key equal to k. If the search is successful, we distinguish between two cases (of increasing difficulty):

- If p has at most one child, the deletion of the node at position p is easily implemented. When introducing update methods for the LinkedBinaryTree class in Section 8.3.1, we declared a nonpublic utility, _delete(p), that deletes a node at position p and replaces it with its child (if any), presuming that p has at most one child. That is precisely the desired behavior. It removes the item with key k from the map while maintaining all other ancestor-descendant relationships in the tree, thereby assuring the upkeep of the binary search tree property. (See Figure 11.5.)
- If position p has two children, we cannot simply remove the node from T since this would create a "hole" and two orphaned children. Instead, we proceed as follows (see Figure 11.6):
  - We locate position r containing the item having the greatest key that is strictly less than that of position p, that is, r = before(p) by the notation of Section 11.1.1. Because p has two children, its predecessor is the rightmost position of the left subtree of p.
  - We use r's item as a replacement for the one being deleted at position p. Because r has the immediately preceding key in the map, any items in p's right subtree will have keys greater than r and any other items in p's left subtree will have keys less than r. Therefore, the binary search tree property is satisfied after the replacement.
  - Having used r's as a replacement for p, we instead delete the node at position r from the tree. Fortunately, since r was located as the rightmost position in a subtree, r does not have a right child. Therefore, its deletion can be performed using the first (and simpler) approach.

As with searching and insertion, this algorithm for a deletion involves the traversal of a single path downward from the root, possibly moving an item between two positions of this path, and removing a node from that path and promoting its child. Therefore, it executes in time $O(h)$ where h is the height of the tree.

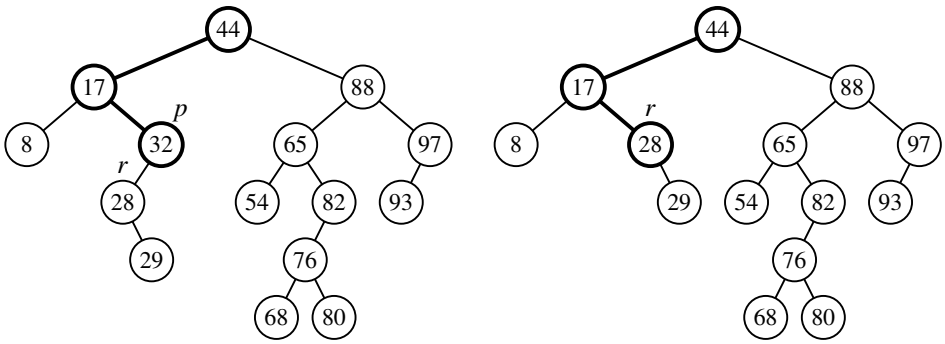


Figure 11.5: Deletion from the binary search tree of Figure 11.4b, where the item to delete (with key 32) is stored at a position p with one child r: (a) before the deletion; (b) after the deletion.

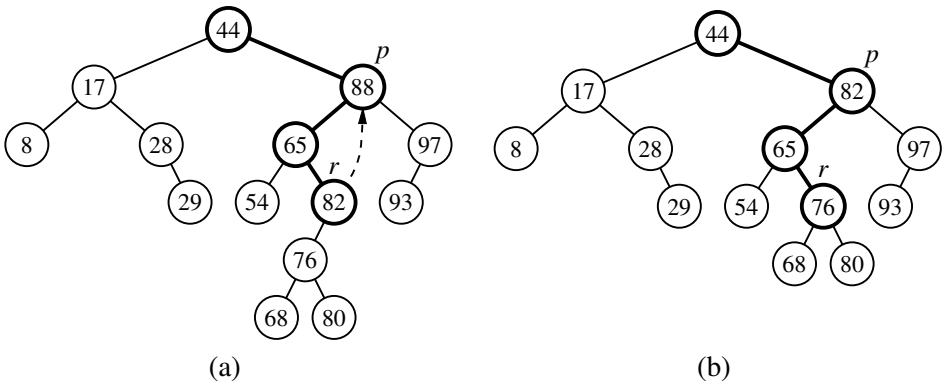


Figure 11.6: Deletion from the binary search tree of Figure 11.5b, where the item to delete (with key 88) is stored at a position p with two children, and replaced by its predecessor r: (a) before the deletion; (b) after the deletion.

11.1.4 Python Implementation

In Code Fragments 11.4 through 11.8 we define a TreeMap class that implements the sorted map ADT using a binary search tree. In fact, our implementation is more general. We support all of the standard map operations (Section 10.1.1), all additional sorted map operations (Section 10.3), and positional operations including first( ), last( ), find_position(k), before(p), after(p), and delete(p).

Our TreeMap class takes advantage of ***multiple inheritance*** for code reuse, inheriting from the LinkedBinaryTree class of Section 8.3.1 for our representation as a positional binary tree, and from the MapBase class from Code Fragment 10.2 of Section 10.1.4 to provide us with the key-value composite item and the concrete behaviors from the collections.MutableMapping abstract base class. We subclass the nested Position class to support more specific p.key( ) and p.value( ) accessors for our map, rather than the p.element( ) syntax inherited from the tree ADT.

We define several nonpublic utilities, most notably a _subtree_search(p, k) method that corresponds to the TreeSearch algorithm of Code Fragment 11.2. That returns a position, ideally one that contains the key k, or otherwise the last position that is visited on the search path. We rely on the fact that the final position during an unsuccessful search is either the nearest key less than k or the nearest key greater than k. This search utility becomes the basis for the public find_position(k) method, and also for internal use when searching, inserting, or deleting items from a map, as well as for the robust searches of the sorted map ADT.

When making structural modifications to the tree, we rely on nonpublic update methods, such as _add_right, that are inherited from the LinkedBinaryTree class (see Section 8.3.1). It is important that these inherited methods remain nonpublic, as the search tree property could be violated through misuse of such operations.

Finally, we note that our code is peppered with calls to presumed methods named _rebalance_insert, _rebalance_delete, and _rebalance_access. These methods serve as ***hooks*** for future use when balancing search trees; we discuss them in Section 11.2. We conclude with a brief guide to the organization of our code.

Code Fragment 11.4: Beginning of TreeMap class including redefined Position class and nonpublic search utilities.

Code Fragment 11.5: Positional methods first( ), last( ), before(p), after(p), and find_position(p) accessor.

Code Fragment 11.6: Selected methods of the sorted map ADT: find_min( ), find_ge(k), and find_range(start, stop); related methods are omitted for the sake of brevity.

Code Fragment 11.7: __getitem__(k), __setitem__(k, v), and __iter__( ).

Code Fragment 11.8: Deletion either by position, as delete(p), or by key, as __delitem__(k).

```
class TreeMap(LinkedBinaryTree, MapBase):
  """Sorted map implementation using a binary search tree."""

  #---------------------------- override Position class ----------------------------
  class Position(LinkedBinaryTree.Position):
    def key(self):
      """Return key of map's key-value pair."""
      return self.element()._key

    def value(self):
      """Return value of map's key-value pair."""
      return self.element()._value

  #------------------------------ nonpublic utilities ------------------------------
  def _subtree_search(self, p, k):
    """Return Position of p's subtree having key k, or last node searched."""
    if k == p.key():                                  # found match
      return p
    elif k < p.key():                                 # search left subtree
      if self.left(p) is not None:
        return self._subtree_search(self.left(p), k)
    else:                                             # search right subtree
      if self.right(p) is not None:
        return self._subtree_search(self.right(p), k)
    return p                                          # unsucessful search

  def _subtree_first_position(self, p):
    """Return Position of first item in subtree rooted at p."""
    walk = p
    while self.left(walk) is not None:                # keep walking left
      walk = self.left(walk)
    return walk

  def _subtree_last_position(self, p):
    """Return Position of last item in subtree rooted at p."""
    walk = p
    while self.right(walk) is not None:               # keep walking right
      walk = self.right(walk)
    return walk
```

Code Fragment 11.4: Beginning of a TreeMap class based on a binary search tree.

```
40    def first(self):
41      """Return the first Position in the tree (or None if empty)."""
42      return self._subtree_first_position(self.root( )) if len(self) > 0 else None
43
44    def last(self):
45      """Return the last Position in the tree (or None if empty)."""
46      return self._subtree_last_position(self.root( )) if len(self) > 0 else None
47
48    def before(self, p):
49      """Return the Position just before p in the natural order.
50
51      Return None if p is the first position.
52      """
53      self._validate(p)                       # inherited from LinkedBinaryTree
54      if self.left(p):
55        return self._subtree_last_position(self.left(p))
56      else:
57        # walk upward
58        walk = p
59        above = self.parent(walk)
60        while above is not None and walk == self.left(above):
61          walk = above
62          above = self.parent(walk)
63        return above
64
65    def after(self, p):
66      """Return the Position just after p in the natural order.
67
68      Return None if p is the last position.
69      """
70      # symmetric to before(p)
71
72    def find_position(self, k):
73      """Return position with key k, or else neighbor (or None if empty)."""
74      if self.is_empty( ):
75        return None
76      else:
77        p = self._subtree_search(self.root( ), k)
78        self._rebalance_access(p)             # hook for balanced tree subclasses
79        return p
```

Code Fragment 11.5: Navigational methods of the TreeMap class.

```
80    def find_min(self):
81      """Return (key,value) pair with minimum key (or None if empty)."""
82      if self.is_empty( ):
83        return None
84      else:
85        p = self.first( )
86        return (p.key( ), p.value( ))
87
88    def find_ge(self, k):
89      """Return (key,value) pair with least key greater than or equal to k.
90
91      Return None if there does not exist such a key.
92      """
93      if self.is_empty( ):
94        return None
95      else:
96        p = self.find_position(k)                    # may not find exact match
97        if p.key( ) < k:                             # p's key is too small
98          p = self.after(p)
99        return (p.key( ), p.value( )) if p is not None else None
100
101   def find_range(self, start, stop):
102     """Iterate all (key,value) pairs such that start <= key < stop.
103
104     If start is None, iteration begins with minimum key of map.
105     If stop is None, iteration continues through the maximum key of map.
106     """
107     if not self.is_empty( ):
108       if start is None:
109         p = self.first( )
110       else:
111         # we initialize p with logic similar to find_ge
112         p = self.find_position(start)
113         if p.key( ) < start:
114           p = self.after(p)
115       while p is not None and (stop is None or p.key( ) < stop):
116         yield (p.key( ), p.value( ))
117         p = self.after(p)
```

Code Fragment 11.6: Some of the sorted map operations for the TreeMap class.

```
118   def __getitem__(self, k):
119     """Return value associated with key k (raise KeyError if not found)."""
120     if self.is_empty( ):
121       raise KeyError('Key Error: ' + repr(k))
122     else:
123       p = self._subtree_search(self.root( ), k)
124       self._rebalance_access(p)            # hook for balanced tree subclasses
125       if k != p.key( ):
126         raise KeyError('Key Error: ' + repr(k))
127       return p.value( )
128
129   def __setitem__(self, k, v):
130     """Assign value v to key k, overwriting existing value if present."""
131     if self.is_empty( ):
132       leaf = self._add_root(self._Item(k,v))          # from LinkedBinaryTree
133     else:
134       p = self._subtree_search(self.root( ), k)
135       if p.key( ) == k:
136         p.element( )._value = v              # replace existing item's value
137         self._rebalance_access(p)            # hook for balanced tree subclasses
138         return
139       else:
140         item = self._Item(k,v)
141         if p.key( ) < k:
142           leaf = self._add_right(p, item)  # inherited from LinkedBinaryTree
143         else:
144           leaf = self._add_left(p, item)   # inherited from LinkedBinaryTree
145     self._rebalance_insert(leaf)             # hook for balanced tree subclasses
146
147   def __iter__(self):
148     """Generate an iteration of all keys in the map in order."""
149     p = self.first( )
150     while p is not None:
151       yield p.key( )
152       p = self.after(p)
```

Code Fragment 11.7: Map operations for accessing and inserting items in the TreeMap class. Reverse iteration can be implemented with __reverse__, using symmetric approach to __iter__.

```
153   def delete(self, p):
154     """Remove the item at given Position."""
155     self._validate(p)                       # inherited from LinkedBinaryTree
156     if self.left(p) and self.right(p):      # p has two children
157       replacement = self._subtree_last_position(self.left(p))
158       self._replace(p, replacement.element())       # from LinkedBinaryTree
159       p =  replacement
160     # now p has at most one child
161     parent = self.parent(p)
162     self._delete(p)                         # inherited from LinkedBinaryTree
163     self._rebalance_delete(parent)          # if root deleted, parent is None
164
165   def __delitem__(self, k):
166     """Remove item associated with key k (raise KeyError if not found)."""
167     if not self.is_empty():
168       p = self._subtree_search(self.root(), k)
169       if k == p.key():
170         self.delete(p)                      # rely on positional version
171         return                              # successful deletion complete
172       self._rebalance_access(p)             # hook for balanced tree subclasses
173     raise KeyError('Key Error: ' + repr(k))
```

Code Fragment 11.8: Support for deleting an item from a TreeMap, located either by position or by key.

11.1.5 Performance of a Binary Search Tree

An analysis of the operations of our TreeMap class is given in Table 11.1. Almost all operations have a worst-case running time that depends on h, where h is the height of the current tree. This is because most operations rely on a constant amount of work for each node along a particular path of the tree, and the maximum path length within a tree is proportional to the height of the tree. Most notably, our implementations of map operations __getitem__, __setitem__, and __delitem__ each begin with a call to the _subtree_search utility which traces a path downward from the root of the tree, using $O(1)$ time at each node to determine how to continue the search. Similar paths are traced when looking for a replacement during a deletion, or when computing a position's inorder predecessor or successor. We note that although a single call to the after method has worst-case running time of $O(h)$, the n successive calls made during a call to __iter__ require a total of $O(n)$ time, since each edge is traced at most twice; in a sense, those calls have $O(1)$ amortized time bounds. A similar argument can be used to prove the $O(s+h)$ worst-case bound for a call to find_range that reports s results (see Exercise C-11.34).

Operation	Running Time
k in T	$O(h)$
T[k], T[k] = v	$O(h)$
T.delete(p), del T[k]	$O(h)$
T.find_position(k)	$O(h)$
T.first( ), T.last( ), T.find_min( ), T.find_max( )	$O(h)$
T.before(p), T.after(p)	$O(h)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(h)$
T.find_range(start, stop)	$O(s+h)$
iter(T), reversed(T)	$O(n)$

Table 11.1: Worst-case running times of the operations for a TreeMap T. We denote the current height of the tree with h, and the number of items reported by find_range as s. The space usage is $O(n)$, where n is the number of items stored in the map.

A binary search tree T is therefore an efficient implementation of a map with n entries only if its height is small. In the best case, T has height $h = \lceil \log(n+1) \rceil - 1$, which yields logarithmic-time performance for all the map operations. In the worst case, however, T has height n, in which case it would look and feel like an ordered list implementation of a map. Such a worst-case configuration arises, for example, if we insert items with keys in increasing or decreasing order. (See Figure 11.7.)

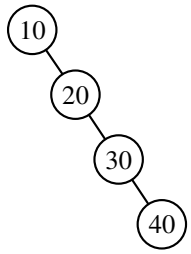


Figure 11.7: Example of a binary search tree with linear height, obtained by inserting entries with keys in increasing order.

We can nevertheless take comfort that, on average, a binary search tree with n keys generated from a random series of insertions and removals of keys has expected height $O(\log n)$; the justification of this statement is beyond the scope of the book, requiring careful mathematical language to precisely define what we mean by a random series of insertions and removals, and sophisticated probability theory.

In applications where one cannot guarantee the random nature of updates, it is better to rely on variations of search trees, presented in the remainder of this chapter, that guarantee a ***worst-case*** height of $O(\log n)$, and thus $O(\log n)$ worst-case time for searches, insertions, and deletions.